

2. Programowanie w REDIS

Programowanie w Redis opiera się na języku skryptowym Lua. Aby swobodnie komunikować się z bazą danych musimy zapoznać się z podstawowymi komendami w tym języku. Zaczniemy od poleceń związanych bezpośrednio z kluczem i wartością kluczową.

2.1. Podstawowe polecenia związane z kluczem

<i>Polecenie</i>	<i>Opis polecenia</i>
SET key	nadanie wartości kluczowi
GET key	odczytanie wartości kluczowej
DEL key	usuwanie wartości kluczowej jeśli istnieje
DUMP key	pobranie serializowanej wersji wartości dla klucza
EXISTS key	sprawdzenie czy istnieje wartość kluczowa
EXPIRE key	ustanawianie czasu po jakim klucz oraz wartość kluczowa przestają istnieć
EXPIREAT key timestamp	jak powyżej, czas podawany w formacie Unix
KEYS pattern	znajdowanie wszystkich kluczy pasujących do wzorca
MOVE key db	przeniesienie klucza do innej bazy danych
PERSIST key	wyłączenie wygasania klucza
PTTL key	zwrócenie czasu, który pozostał do wygaśnięcia klucza w milisekundach
TTL key	zwrócenie czasu, który pozostał do wygaśnięcia klucza
RANDOMKEY	zwrócenie losowego klucza z bazy danych

RENAME key newkey	zmiana wartości klucza
--------------------------	------------------------

TYPE key	odczytanie typu wartości związanej z kluczem
-----------------	--

Wypróbujmy poznane polecenia stosując je odpowiednio w przykładach.

Ćwiczenie 1.

Użyjmy polecenia, które kluczowi o nazwie *nazwisko* nada wartość “Kowalski”.

```
redis 127.0.0.1:6379> SET nazwisko Kowalski
OK
```

Przy użyciu polecenia **GET** sprawdź wartość klucza **nazwisko**.

```
redis 127.0.0.1:6379> GET nazwisko
"Kowalski"
```

Usuń klucz “nazwisko”. Jeśli polecenie zakończy się prawidłowo otrzymamy odpowiedź (integer) 1, w przeciwnym wypadku będzie (integer) 0.

```
redis 127.0.0.1:6379> DEL nazwisko
(integer) 1
```

Ćwiczenie 2.

Od wersji 2.6.12 wprowadzono możliwość ustawienia czasu, po którym informacja znajdująca się pod danym kluczem straci ważność. Jeżeli dysponujemy danymi, które są istotne tylko przez krótki okres czasu, możemy w ten sposób regulować ich „żywołność”. Mogą to być różnego rodzaju wiadomości, posty czy komunikaty, które po upływie czasu, naturalnie tracą ważność. W ten sposób zwalniana jest przestrzeń w pamięci operacyjnej, która jest kluczowa dla bazy danych Redis.

Operację wygaszania można wykonać na dwa sposoby. Przy definiowaniu wartości związanej z kluczem:

```
SET key value [EX seconds] [PX milliseconds] [NX|XX]
```

gdzie EX oznacza liczbę sekund, po której klucz straci swoją ważność, PX liczbę wyrażoną w milisekundach, NX ustawia wartość jeśli klucz nie był zdefiniowany, XX ustawia wartość tylko jeśli klucz był wcześniej zdefiniowany. Drugim sposobem jest użycie polecenia EXPIRE (lub EXPIREAT) dla już istniejącego klucza.

```
EXPIRE key seconds
```

Kluczowi **nazwisko** nadaj wartość **Nowak**, którego długość życia będzie wynosiła 50 sekund.

```
redis 127.0.0.1:6379> SET nazwisko Nowak EX 50
OK
```

Możemy użyć polecenia **TTL** w celu sprawdzenia ile czasu pozostało do wygaśnięcia klucza.

```
redis 127.0.0.1:6379> TTL nazwisko
(integer) 32
```

Przy użyciu polecenia **GET** sprawdź jeszcze raz klucz **nazwisko**. (Jeśli upłynęło 50 sekund klucz ten straci swoją ważność).

```
redis 127.0.0.1:6379> GET nazwisko
(nil)
```

Jeśli chcemy zapobiec wygaśnięciu klucza można użyć polecenia **PERSIST**, jednak trzeba to zrobić zanim straci swoją ważność.

Ćwiczenie 3.

Bardzo przydatnym poleceniem jest **KEYS**, znajdującym wśród kluczy te pasujące do wzorca. Naszym zadaniem jest znaleźć wszystkie klucze związane z nazwiskami. W tym celu zbudujemy wzorzec **nazwisko***. Symbol “?” oznacza dowolny, jeden znak we wzorcu, natomiast symbol * oznacza dowolną ilość znaków występującą we wzorcu.

```
127.0.0.1:6379> set nazwisko1 Nowak
OK
127.0.0.1:6379> set nazwisko2 Kowalski
OK
127.0.0.1:6379> set nazwisko3 Kowalczyk
OK
127.0.0.1:6379> keys nazw*
1) "nazwisko3"
2) "nazwisko1"
3) "nazwisko2"
```

Zadanie 1

Wykorzystując narzędzie klienta Redis wykonaj następujące operacje:

- Utwórz klucz **nazwisko** i nadaj mu wartość Twojego nazwiska. Klucz ma być ważny 150 sekund.
- Sprawdź czy został poprawnie utworzony użyj poleceń (GET i EXISTS).
- Sprawdź jego typ.
- Sprawdź jego żywotność.
- Zapobiegij wygaśnięciu klucza.

2.2.TYPY DANYCH

Redis dostarcza rozbudowany system typów danych, na których można operować. Zapoznamy się teraz z każdym z nich.

2.2.1.Ciągi znaków (Strings)

Ciągi znaków (*ang.Strings*) są podstawowym typem danych w Redis. Ciągi tekstowe są binarnie bezpieczne (*ang.Binary safe*), co oznacza, że Redis nie analizuje zawartości przekazywanych mu danych. Dostarczają wiele możliwości ich wykorzystania. Mogą przechowywać do 512MB danych.

Podstawowe polecenia dla ciągów znaków

<i>Polecenie</i>	<i>Opis polecenia</i>
GETRANGE key start end	zwraca ciąg znaków składowany w kluczu od pozycji start do end
-----	-----
	-
GETSET key value	ustawia nową wartość kluczową i zwraca wartość poprzednią
-----	-----
	-
MGET key1 [key2...]	zwraca wartości dla wszystkich podanych kluczy
-----	-----
	-
STRLEN key	zwraca długość wartości kluczowej
-----	-----
MSET key value [key value...]	nadaje wartości dla wielu kluczy jednocześnie
-----	-----
MSETNX key value [key value...]	nadaje wartości dla wielu kluczy jednocześnie, tylko dla istniejących kluczy
-----	-----
INCR key	zwiększenie wartości klucza o 1
-----	-----
INCRBY key increment	zwiększenie wartości klucza o podaną wartość
-----	-----
INCRBYFLOAT key increment	zwiększenie wartości klucza o podaną wartość, również ułamkową
-----	-----
DECR key	zmniejszanie wartości klucza o 1
-----	-----
DECRBY key decrement	zmniejszanie wartości klucza o podaną wartość
-----	-----
APPEND key value	dołączenie dowolnej wartości do klucza
-----	-----

W tabelce zostały przedstawione tylko wybrane polecenia, pełną ich listę można znaleźć pod adresem: <https://redis.io/commands>.

Ćwiczenie 4

Utworzymy klucz zawierający ciąg znaków 'ALA ma kota i jest on biały'.

```
127.0.0.1:6379> set string 'ALA ma kota i jest on biały'  
OK
```

Przy użyciu polecenia **GETRANGE** możemy zwrócić tylko określoną część ciągu znaków, wyznaczoną przez pozycję startową i końcową.

```
127.0.0.1:6379> getrange string 0 10  
"ALA ma kota"
```

Poleceniem **GETSET** zmienimy zawartość klucza string na wartość 10

```
127.0.0.1:6379> getset string 10  
"ALA ma kota i jest on biały"  
127.0.0.1:6379> get string  
"10"
```

Jeśli wartość klucza jest wartością liczbową można ją zwiększać bądź zmniejszać używając poleceń z rodziny **INCR** i **DECR**.

```
127.0.0.1:6379> decr string  
(integer) 9  
127.0.0.1:6379> decr string  
(integer) 8  
127.0.0.1:6379> decr string  
(integer) 7  
127.0.0.1:6379> incr string  
(integer) 8  
127.0.0.1:6379> incrbyfloat string 0.5  
"8.5"
```

Za pomocą poleceń **MSET** i **MGET**, można jednocześnie operować na wielu kluczach.

```
127.0.0.1:6379> mset a 10 b 20 c 30  
OK  
127.0.0.1:6379> mget a b c  
1) "10"  
2) "20"  
3) "30"
```

Zadanie 2

Dla utrwalenia wiadomości wykonaj samodzielnie:

- Wybierz losowy klucz z Twojej bazy danych.
- Sprawdź długość jego wartości.

- Doklej do niego ciąg znaków 'RAZ DWA RAZ DWA RAZ DWA RAZ DWA RAZ DWA'.
- Ustaw automatyczne wygasanie po 10 sekundach.
- Sprawdź czy został usunięty z bazy danych.

2.2.2.Listy (Lists)

Listy w Redis są w rzeczywistości listami ciągów znaków. Dzięki temu dostęp do danych może odbywać się w takiej samej kolejności w jakiej zostały dodane. Zaletą list jest ten sam czas dodania elementu zarówno na początku jak i na końcu listy niezależnie od rozmiaru listy. Maksymalna długość listy wynosi $2^{32} - 1$ czyli powyżej 4mld elementów.

Podstawowe polecenia dla list:

<i>Polecenie</i>	<i>Opis polecenia</i>
LINDEX key index	zwraca element z listy według indeksu
LINSERT key BEFORE/AFTER pivot value	wstawia element przed lub po wybranym elemencie z listy
LLEN key	zwraca długość listy
LPOP key	usuwa i zwraca usuwany element z listy
LPUSH key value1 [value2]	wstawia jeden bądź wiele elementów do listy
LPUSHX key value	wstawia element do listy tylko jeśli lista istnieje
LRANGE key start stop	zwraca fragment listy od pozycji start do stop
LREM key count value	usuwa element z listy
LSET key index value	wstawia element Sets the value of an element in a list by its index
LTRIM key start stop	przycina listę do ograniczonej elementami start i stop
RPOP key	usuwa i zwraca ostatni element z listy
RPOPLPUSH source destination	przenosi ostatni element pomiędzy listami i zwraca go

RPUSH key value1 [value2]

dodaje jeden bądź wiele elementów do listy

RPUSHX key value

dodaje element do listy jeśli lista istnieje

Ćwiczenie 5

Utworzymy listę o nazwie 'my_list' zawierającą elementy: Oracle MySql MongoDB Redis DB2

```
127.0.0.1:6379> lpush my_list Oracle MySql MongoDB Redis DB2
(integer) 5
```

Kolejne elementy możemy dodać zarówno na początek jak i koniec listy, wykorzystujemy do tego odpowiednio polecenie **LPUSH** (początek listy) i **RPUSH** (koniec listy). Do naszej listy, na jej koniec dodajmy element SQLite.

```
127.0.0.1:6379> rpush my_list SQLite
(integer) 6
```

Możemy wyświetlić zawartość listy bądź jej fragment korzystając z polecenia **LRANGE**. Podajemy wartości brzegowe listy, które nas interesują.

```
127.0.0.1:6379> lrange my_list 0 10
1) "DB2"
2) "Redis"
3) "MongoDb"
4) "MySql"
5) "Oracle"
6) "SQLite"
```

Jeśli chcemy wstawić element listy na konkretnej pozycji, możemy użyć polecenia **LSET**. Wstawimy element 'Cassandra' na początek listy.

```
127.0.0.1:6379> lset my_list 0 Cassandra
OK
```

```
127.0.0.1:6379> lrange my_list 0 10
1) "Cassandra"
2) "Redis"
3) "MongoDb"
4) "MySql"
5) "Oracle"
6) "SQLite"
```

Zadanie 3

- Wyświetl wszystkie klucze, które do tej pory zostały utworzone w bazie danych (wskazówka: **KEYS**).
- Wybierz taki, który jest listą i dodaj do niej 10 elementów.

- Sprawdź ile elementów posiada lista.
- Przytnij wybraną listę do 5 pierwszych elementów.
- Usuń pierwszy i ostatni element z listy.
- Utwórz nową listę nowa_lista i dodaj do niej kilka elementów.
- Przenieś ostatni element listy nowa_lista do poprzedniej listy.

2.2.3. Tablice asocjacyjne (Hashes)

Tablice asocjacyjne sprawdzają się do reprezentowania obiektów w bazie danych Redis. Dowolny obiekt możemy mapować na zestaw pól i wartości. Tablice mogą być wykorzystane do reprezentacji połączonych ze sobą struktur danych za pomocą referencji.

<i>Polecenie</i>	<i>Opis polecenia</i>
HDEL key field2 [field2]	usunięcie jednej lub wielu par związanych z kluczem
HEXISTS key field	sprawdzenie, czy klucz zawiera podane pole
HGET key field	zwraca wartość konkretnego pola dla podanego klucza
HGETALL key	odczytanie wszystkich par pole-wartość dla podanego klucza
HINCRBY key field increment	zwiększenie wartości (całkowitej) dla danego pola związanego z kluczem
HINCRBYFLOAT key field increment	zwiększenie wartości (float) dla danego pola związanego z kluczem
HKEYS key	pobranie wszystkich pól dla podanego klucza
HLEN key	zwraca liczbę pól dla podanego klucza
HMGET key field1 [field2]	odczytanie wartości dla wskazanego pola związanego z kluczem
HMSET key field1 value1 [field2 value2]	zapisanie klucza z kolekcją par pole-wartość
HSET key field value	ustawia wartość dla podanego pola związanego z

	kluczem
HSETNX key field value	Sustawia wartość dla podanego pola związanego z kluczem jeśli klucz istnieje
HVALS key	pobranie wszystkich wartości dla podanego klucza

Ćwiczenie 6

Stworzymy teraz kolekcję książek korzystając z tablicy asocjacyjnej. Wstawiamy trzy klucze powiązane z kolekcjami pole-wartość.

klucz: '83-7197-669-0'
pole tytuł: 'Access 2002. Projektowanie baz danych. Księga eksperta',
pole cena: 79.0,
pole wydawnictwo: 'Helion',
pole rok: 2002

klucz: '83-7197-786-7',
pole tytuł: 'Access 2002/XP PL dla każdego',
pole cena: 65.0,
pole wydawnictwo: 'Helion',
pole rok: 2003

klucz: '83-7279-149-X',
pole tytuł: 'Dane w sieci WWW',
pole cena: 31,
pole wydawnictwo: 'Mikom',
pole rok: 2001

```
127.0.0.1:6379> hmset 83-7197-669-0 tytuł 'Access 2002. Projektowanie baz dan
ych. Księga eksperta' cena 79.0 wydawnictwo Helion rok 2002
OK
127.0.0.1:6379> hmset 83-7197-786-7 tytuł 'Access 2002/XP PL dla każdego' cen
a 65.0 wydawnictwo Helion rok 2003
OK
127.0.0.1: 6379> hmset 83-7279-149-X tytuł 'Dane w sieci WWW' cena 31.0 wydaw
nictwo Mikom rok 2001
OK
```

Możemy wyświetlić wszystkie pary pole-wartość dla klucza '83-7279-149-X' używając polecenia **HGETALL**.

```
127.0.0.1:6379> hgetall 83-7279-149-X
1) "tytuł"
2) "Dane w sieci WWW"
3) "cena"
```

- 4) "31.0"
- 5) "wydawnictwo"
- 6) "Mikom"
- 7) "rok"
- 8) "2001"

A jeśli interesuje nas tylko jedno konkretne pole (np cena książki), korzystamy z polecenia **HGET**

```
127.0.0.1:6379> hget 83-7279-149-X cena  
"31.0"
```

Możemy również dostać się do większej ilości pól jednocześnie, korzystając z polecenie **HMGET**.

```
127.0.0.1:6379> hmget 83-7279-149-X cena wydawnictwo  
1) "31.0"  
2) "Mikom"
```

Jak również zmodyfikować konkretne pole poleceniem **HSET**.

```
127.0.0.1:6379> hset 83-7279-149-X cena 75  
OK
```

```
127.0.0.1:6379> hgetall 83-7279-149-X  
1) "tytul"  
2) "Dane w sieci WWW"  
3) "cena"  
4) "75"  
5) "wydawnictwo"  
6) "Mikom"  
7) "rok"  
8) "2001"
```

Zadanie 4

Zbuduj kolekcję dla 3 pracowników:

klucz: '100',
pole nazwisko: 'WEGLARZ',
pole stanowisko: dyrektor,
pole placa_pod: 1730,00,
pole placa_dod: 420,50,
pole data_zatrudnienia: 01-01-1968,
pole id_zesp: 10

klucz: '110',
pole nazwisko: 'BLAZEWICZ',
pole stanowisko: profesor,
pole placa_pod: 1350,00,
pole placa_dod: 210,00,

pole data_zatrudnienia: 01-05-1973,
pole id_zesp: 40

klucz: '120',
pole nazwisko: 'SLOWINSKI',
pole stanowisko: profesor,
pole placa_pod: 1070,00,
pole placa_dod: NULL,
pole data_zatrudnienia: 01-09-1977,
pole id_zesp: 30

- Wyświetl nazwy wszystkich pól należących do klucza 100.
- Kluczowi 110 zwiększ płacę podstawową o 250,50.
- Usuń pracownika o kluczu 120
- Wyświetl wszystkie klucze rozpoczynające się od '1'

2.2.4 Zbiory (Sets)

Zbiór jest kolekcją ciągów znaków. Zbiory zawierają unikalne i nieuporządkowane wartości. Dodawanie, usuwanie czy wyszukiwanie elementu ma stałą złożoność obliczeniową równą $O(1)$. Czyli jeśli mamy kolekcję, którą będziemy często modyfikować bądź przeszukiwać powinniśmy użyć zbiór do przechowania danych.

<i>Polecenie</i>	<i>Opis polecenia</i>
SADD key member1 [member2]	dodanie elementu do zbioru dla podanego klucza
SCARD key	pobranie liczby elementów zbioru dla podanego klucza
SDIFF key1 [key2]	wyznaczenie różnicy zbiorów
SDIFFSTORE destination key1 [key2]	wyznaczenie różnicy dla wielu zbiorów
SINTER key1 [key2]	wyznaczenie części wspólnej zbiorów
SINTERSTORE destination key1 [key2]	wyznaczenie części wspólnej dla wielu zbiorów
SMEMBERS key	pobranie wszystkich elementów zbioru dla podanego klucza
SISMEMBERS key member	sprawdzenie, czy element należy do zbioru

SMOVE source destination member	przenoszenie elementu z jednego do drugiego zbioru
SPOP key	pobranie i usunięcie losowo wybranego elementu zbioru dla podanego klucza
SRANDMEMBER key [count]	zwraca jeden/wiele losowych elementów ze zbioru
SREM key member1 [member2]	usunięcie elementu(ów) zbioru dla podanego klucza
SUNION key1 [key2]	wyznaczenie sumy zbiorów
SUNIONSTORE dest key1 [key2]	wyznaczenie sumy dla wielu zbiorów

Ćwiczenie 7

Chcemy utworzyć zbiór posiadający kilka adresów IP (np. zbiór osób odwiedzających naszą stronę internetową)

```
127.0.0.1:6379> sadd my_page 150.254.30.30 150.254.5.4
(integer) 2
127.0.0.1:6379> sadd my_page 32.44.143.42 31.142.34.45
(integer) 2
127.0.0.1:6379> sadd my_page 150.254.30.30 31.142.34.46
(integer) 1

127.0.0.1:6379> smembers my_page
1) "150.254.30.30"
2) "31.142.34.45"
3) "150.254.5.4"
4) "32.44.143.42"
5) "31.142.34.46"
```

Chcemy również posiadać informacje o adresach IP odwiedzających naszego bloga, w tym celu tworzymy drugi zbiór my_blog.

```
127.0.0.1:6379> sadd my_blog 150.254.35.35 150.254.50.45
(integer) 2
127.0.0.1:6379> sadd my_blog 150.254.30.30 31.142.34.46
(integer) 2

127.0.0.1:6379> smembers my_blog
1) "150.254.50.45"
2) "31.142.34.46"
```

- 3) "150.254.30.30"
- 4) "150.254.35.35"

Posiadamy dwa zbiory z informacją o adresach IP odwiedzającymi nasze witryny. Możemy za pomocą polecenia **SUNION** lub **SUNIONSTORE** utworzyć zbiór będący częścią wspólną naszych dwóch zbiorów.

```
127.0.0.1:6379> sunionstore my_sites my_blog my_page
(integer) 7
127.0.0.1:6379> smembers my_sites
1) "32.44.143.42"
2) "150.254.5.4"
3) "31.142.34.46"
4) "31.142.34.45"
5) "150.254.50.45"
6) "150.254.30.30"
7) "150.254.35.35"
```

A jeśli interesuje nas ile osób jednocześnie odwiedzało naszą stronę i naszego bloga możemy użyć polecenia **SINTER**, do wyznaczenia części wspólnej naszych zbiorów.

```
127.0.0.1:6379> SINTER my_page my_blog
1) "31.142.34.46"
2) "150.254.30.30"
```

Rozbudowany zestaw poleceń do manipulowania zbiorami sprawia, że szybko i łatwo możemy na nich operować.

Zadanie 5

Zbuduj własny zbiór zawierający 10 adresów IP. Dodaj do niego adres "150.254.30.30". Sprawdź działanie wszystkich poleceń działających na wielu zbiorach wykorzystując utworzone wcześniej zbiory.

2.2.5 Zbiory uporządkowane (Sorted Sets)

Zbiory posortowane są ulepszoną wersją zbiorów. Jak sama nazwa sugeruje są uporządkowane (posortowane). Każdy element uporządkowanego zbioru ma przydzielony numer (score), który pozwala na posortowanie zbioru od najmniejszego do największego. Możemy szybko manipulować elementami posortowanego zbioru, ponieważ elementy są od razu wstawiane w odpowiedniej kolejności, zamiast sortowania po wstawieniu. Ze zbiorami posortowanymi jest związana specjalna grupa poleceń, która ma przedrostek **Z**. Pełną listę poleceń dla zbiorów posortowanych znajdziemy na stronie: https://redis.io/commands#sorted_set

2.2.6 Struktura HyperLogLog

HyperLogLog jest probabilistyczną strukturą danych przeznaczoną do szybkiego zliczania unikalnych wartości w zbiorze przy użyciu niewielkich zasobów pamięciowych (ok 12 kb/key) z błędem standardowym < 0.1 %. Struktura może przechowywać 2^{64} elementów.

<i>Polecenie</i>	<i>Opis polecenia</i>
PFADD key element [element ...]	dodaje element do struktury HyperLogLog.
PFCOUNT key [key ...]	zwraca licznosc zbioru
PFMERGE destkey sourcekey [sourcekey ...]	scala N roznych struktur HyperLogLog w jedna strukture

Zadanie 6

Zbuduj strukture HyperLogLog, ktora bedzie zawierala 100000 elementow. Zwróc licznosc stworzonej struktury.

Transakcje w Redis

Transakcje z baze Redis sa rozumiane jak wykonanie grupy polecen w jednym kroku. Wspierane sa tylko wlasnosci izolacji i atomowosci transakcji. Wszystkie polecenia w transakcji sa realizowane jako pojedyncza izolowana operacja. Jednoczesna modyfikacja nie jest mozliwa przez inną operacje. Transakcja jest atomowa, albo wykonane sa wszystkie operacje, albo wszystkie sa wycofane.

Polecenia związane z transakcjami:

<i>Polecenie</i>	<i>Opis polecenia</i>
DISCARD	anulowanie wszystkich operacji wchodzących w skład transakcji (do polecenia MULTI)
EXEC	wykonanie wszystkich poleceń w ramach transakcji rozpoczętej poleceniem MULTI
MULTI	start transakcji
UNWATCH	zdejmowanie blokad
WATCH key [key ...]	zakładanie blokady na klucz(e)

Ćwiczenie 8

Aby przeanalizować działanie transakcji w Redis stworzymy strukture, która będzie przechowywać dane o zespołach i ich pracownikach. Możemy wykorzystać tablice asocjacyjną do przechowania takich danych.

```
127.0.0.1:6379> hmset ADM10 nazwa 'Administracja' Id 10
OK
127.0.0.1:6379> hmset BD20 nazwa 'Badania operacyjne' Id 20
OK
127.0.0.1:6379> hmset ALG30 nazwa 'Algorytmy' Id 30
OK

127.0.0.1:6379> hmset pracownik1 nazwisko Nowak placa_pod 1000
OK
127.0.0.1:6379> hmset pracownik2 nazwisko Kowalski placa_pod 1200
OK
127.0.0.1:6379> hmset pracownik3 nazwisko Kowalczykiewicz placa_pod 1100
OK
```

Pracownik Nowak jest zatrudniony w zespole 10 na pół etatu, a drugą część etatu realizuje w zespole 20.

```
127.0.0.1:6379> sadd ADM10_prac pracownik1
OK
127.0.0.1:6379> sadd BD20_prac pracownik1 pracownik2
OK
127.0.0.1:6379> sadd ALG30_prac pracownik3
OK

127.0.0.1:6379> smembers ADM10_prac
1) "pracownik1"
127.0.0.1:6379> smembers BD20_prac
1) "pracownik1"
2) "pracownik2"
```

Jeśli chcielibyśmy zwolnić pracownika o nazwisku Nowak, aby zachować spójność musielibyśmy przeglądać wszystkie elementy typu: Zespół_prac, aby znaleźć wszystkie wystąpienia pracownik1 i odpowiednio zaktualizować kolekcję.

Wykorzystamy teraz transakcje do wykonania tej operacji. Transakcje rozpoczynamy poleceniem **MULTI**, natomiast poleceniem **EXEC** kończymy. Do momentu pojawienia się polecenia **EXEC** wszystkie operacje dodawane są do kolejki zadań (status **(QUEUED)**). Polecenie **EXEC** uruchamia wszystkie polecenia z kolejki i kończy transakcję. Jeśli chcielibyśmy wycofać wszystkie polecenia w transakcji (po poleceniu **MULTI**) możemy użyć komendy **DISCARD**. W Redis nie ma możliwości wykonania operacji **rollback**, w takim sensie jak rozumieliśmy go z poziomu relacyjnej bazy danych. Komenda **DISCARD** anuluje wszystkie polecenia wykonane przez transakcję, ale w przypadku gdy pojawi się błąd (awaria) podczas działania transakcji to nie ma możliwości jej wycofania.

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> DEL pracownik1
QUEUED
127.0.0.1:6379> smembers ADM10
QUEUED
```

```
127.0.0.1:6379> DEL ADM10_prac
QUEUED
127.0.0.1:6379> smembers BD20_prac
QUEUED
127.0.0.1:6379> SREM BD20_prac pracownik1
QUEUED
127.0.0.1:6379> smembers BD20_prac
127.0.0.1:6379> EXEC
1) (integer) 1
2) (integer) 1
3) 1) "pracownik1"
   2) "pracownik2"
4) (integer) 1
5) (integer) 1
6) 1) "pracownik2"
```

Ćwiczenie 9

Przyjrzyjmy się działaniu poleceń **WATCH** i **UNWATCH**, które ma na celu zakładanie blokady na wybranym kluczu. Planujemy w ramach transakcji zmodyfikować klucz o nazwie pracownik1, o nazwisku Nowak podnosząc mu pensję o 500. Aby zapewnić spójność i pewność, że dane nie ulegną zmianie, zakładamy blokadę na nasz klucz pracownik1.

```
127.0.0.1:6379> watch pracownik1
OK
```

Jeżeli nasz pracownik zostałby zmodyfikowany przez inną operację.

```
127.0.0.1:6379> hset pracownik1 placa_pod 2000
(integer) 0
```

To po wykonaniu naszej transakcji z podwyżką otrzymalibyśmy następujący wynik operacji:

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> hset pracownik1 placa_pod 1500
QUEUED
127.0.0.1:6379> hgetall pracownik1
QUEUED
127.0.0.1:6379> exec
(nil)
```

Wartość pusta (nil) oznacza, że polecenia w transakcji nie zostały wykonane, ponieważ „obserwowany” klucz uległ modyfikacji. Polecenie **WATCH** nie blokuje zasobu (klucza) przed modyfikacją przez inną operację. Polecenia **UNWATCH** zdejmuję blokadę z klucza.

Zadanie 7

Do zespołu ALG30 przyjmij nowego pracownika (pracownik4). Następnie w ramach transakcji zwiększ mu płace podstawową o 1500. Wycofaj wprowadzone przez transakcję zmiany.